Universidad El Bosque · Ingeniería de Sistemas

Bases de Datos 2 · Docente: Christian Felipe Duarte

SIRAV

Sistema Integral de Reservas y Alquiler de Vehículos: plataforma web con PostgreSQL, MongoDB y despliegue en una máquina virtual Linux

Integrantes
David Felipe Bejarano Montes · José David Rodríguez

Proyecto
Proyecto final · en desarrollo

Actualizado
3 de noviembre de 2026

Contenido

Resumen
1. Problema, justificación y objetivos
2. Alcance, supuestos y límites
3. Arquitectura
4. Funcionamiento y reglas de negocio
5. Diseño de datos
6. Requerimientos
7. Gestión del proyecto
8. Estado actual
Fuentes

Resumen

SIRAV es el proyecto final de Bases de Datos 2: una aplicación web para una empresa pequeña de alquiler de vehículos. Con ella se consultan vehículos, se hacen reservas, se registran entregas y devoluciones y se guarda el historial de cada alquiler. Lo más importante es que el sistema no permite, por diseño, que un mismo vehículo quede asignado a dos períodos que se superponen, y esa regla la hace cumplir la base de datos y no solo la interfaz.

La solución usa dos bases de datos con funciones distintas. PostgreSQL guarda los datos transaccionales y la lógica de negocio (procedimientos, funciones, triggers, excepciones y transacciones). MongoDB guarda las inspecciones de los vehículos y la bitácora de eventos. La aplicación se hace con Spring Boot y todo corre sobre Ubuntu Server en una máquina virtual.

> `Estado del proyecto`
>
> SIRAV está en desarrollo. Esta página reúne lo que ya está definido en la propuesta (versión 1.0, del 17 de octubre de 2026) y el avance que muestra Jira al 3 de noviembre de 2026. El modelo de datos, los scripts de la base, la aplicación y el informe final se irán publicando aquí a medida que estén terminados.

Ficha	
Tipo de proyecto	Aplicación web académica, prototipo funcional con datos de prueba
Equipo	David Felipe Bejarano Montes y José David Rodríguez
Aplicación	Java, Spring Boot, Spring Security, Thymeleaf y Bootstrap
Base relacional	PostgreSQL (PL/pgSQL)
Base documental	MongoDB
Plataforma	Ubuntu Server en máquina virtual
Gestión	Metodología híbrida, cuatro sprints semanales en Jira y repositorio en GitHub

1. Problema, justificación y objetivos

1.1 Problema

Cuando las reservas se llevan en hojas de cálculo, mensajes y registros manuales aparecen las asignaciones duplicadas, la confusión sobre el estado de cada carro, los errores al liquidar el alquiler y la dificultad para consultar el historial de un vehículo. Sin una fuente única de información, además, se demora la atención al cliente y la elaboración de reportes.

1.2 Por qué es un buen caso para Bases de Datos 2

Evitar reservas que se cruzan es un ejemplo claro de integridad y concurrencia: pide restricciones declarativas, transacciones, manejo de excepciones y pruebas con accesos simultáneos. Además, la información del negocio es de dos tipos. Los datos estructurados y transaccionales encajan en PostgreSQL. Los registros de estructura variable, como una inspección con cero o muchos daños, encajan mejor en MongoDB. Eso justifica usar las dos bases de forma complementaria.

1.3 Objetivos

Objetivo general. Centralizar la información de clientes, vehículos y alquileres en una aplicación web que controle la disponibilidad, registre los movimientos de forma confiable y conserve un historial consultable, combinando PostgreSQL y MongoDB sobre un entorno Linux virtualizado y con una gestión formal del proyecto.

Objetivos específicos:

- Modelado: modelo E-R (Chen), E-R extendido, modelo relacional en Crow's Foot y diccionario de datos, normalizados hasta 3FN.
- Lógica en la base de datos: procedimientos, funciones, triggers, excepciones y transacciones en PL/pgSQL.
- Base documental: CRUD, consultas complejas y agregaciones en MongoDB, con justificación de qué datos van allí.
- Integración y despliegue: aplicación Spring Boot conectada a ambas bases y desplegada en una VM Ubuntu Server documentada.
- Gestión: cronograma, requerimientos, riesgos, metodología y control de versiones en GitHub.

2. Alcance, supuestos y límites

Tipo	Supuesto
Negocio	Una sola sede y una flota de prueba de 20 a 30 vehículos en 4 o 5 categorías. Cada reserva es de un cliente y un vehículo.
Negocio	Tarifas, pagos y cargos son valores de demostración en pesos colombianos, sin pasarela de pago.
Negocio	Las fotos de inspección se referencian por nombre de archivo, no se guardan en la base. Hay tres perfiles de usuario: administrador, operador y cliente.
Técnico	PostgreSQL es la fuente única de verdad para reservas, alquileres y pagos. MongoDB guarda inspecciones y bitácora, enlazadas por el identificador del alquiler.
Técnico	La aplicación y las dos bases corren en una VM Ubuntu Server (recomendado: 6 GB de RAM, 2 núcleos y 40 GB de disco). Todo el software es gratuito o de código abierto.
Gestión	Dos integrantes, plazo de cuatro semanas, cuatro sprints semanales en Jira y alcance congelado con la propuesta.

> Lo que queda fuera
>
> Para llegar al plazo se excluyen los pagos bancarios reales, la geolocalización en tiempo real, la firma electrónica con validez jurídica, la verificación externa de licencias, los seguros comerciales, una API REST con interfaz independiente y la replicación o fragmentación de MongoDB. Quedan como ampliaciones futuras.

3. Arquitectura

La aplicación presenta el catálogo y los formularios con Thymeleaf y Bootstrap. Desde Spring Boot se llaman los procedimientos de PostgreSQL y se consulta MongoDB, y todo se ejecuta en un servidor Ubuntu virtualizado al que se entra desde el navegador del equipo anfitrión.

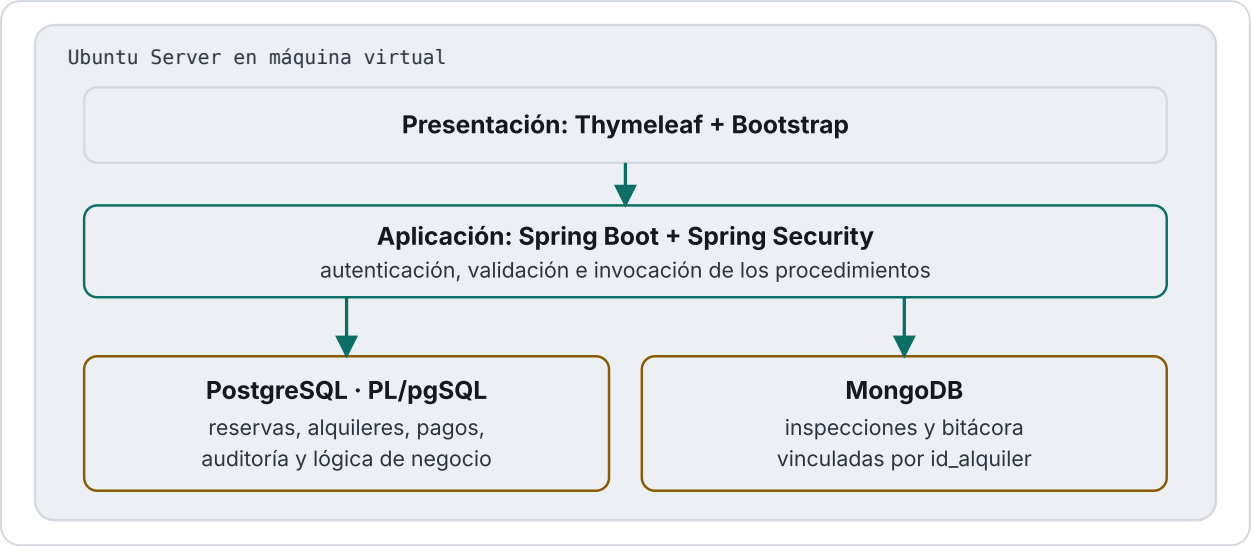


Figura 1 · Capas de SIRAV. Elaboración propia a partir de la propuesta del proyecto.

Capa	Tecnología	Responsabilidad
Presentación	Thymeleaf y Bootstrap	Interfaz web para clientes y personal
Aplicación	Java, Spring Boot, Spring Security	Autenticación, validación de entradas, invocación de procedimientos (`SimpleJdbcCall` o `JdbcTemplate`) y acceso a MongoDB con Spring Data
Datos relacionales	PostgreSQL, PL/pgSQL	Reservas, disponibilidad, alquileres, tarifas, pagos de prueba, auditoría y lógica de negocio
Datos documentales	MongoDB	Inspecciones, observaciones, evidencias referenciadas y bitácora de eventos
Infraestructura	Ubuntu Server en VM	Aloja la aplicación y las dos bases de datos

4. Funcionamiento y reglas de negocio

4.1 Módulos

Módulo	Funcionalidad
Usuarios y perfiles	Inicio de sesión y diferenciación entre administrador, operador y cliente
Catálogo de vehículos	Consulta por categoría, características, tarifa diaria, estado y fechas libres
Reservas	Selección de fechas, validación de disponibilidad y confirmación o cancelación
Alquileres	Entrega, fechas efectivas, inspección y condiciones de devolución
Mantenimiento	Períodos en los que un vehículo no se puede reservar
Cobros simulados	Estimación del costo, pagos de prueba y cargos por retraso
Historial y reportes	Alquileres, ocupación, incidencias y reportes SQL y de MongoDB

4.2 Ciclo de una reserva

Una reserva avanza por estados en un orden fijo. Cada cambio lo hace un procedimiento almacenado, y la aplicación solo lo invoca.

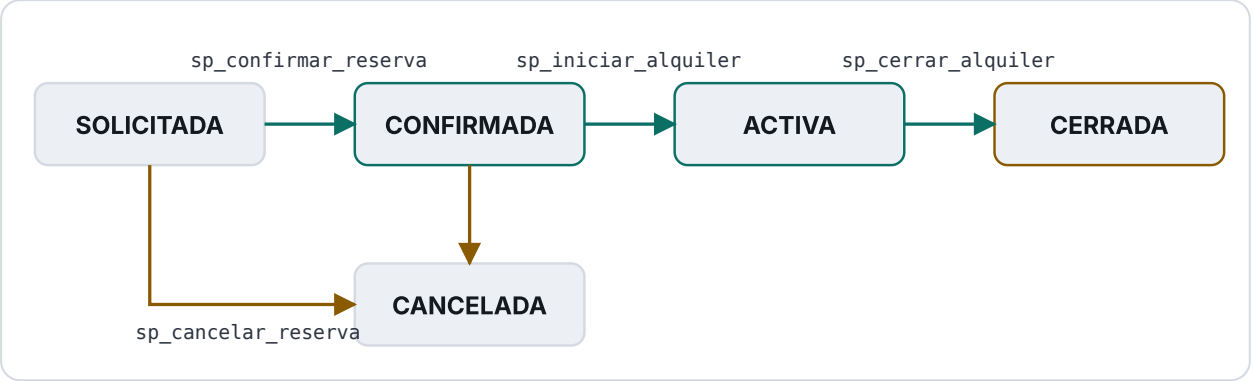


Figura 2 · Estados de una reserva y el procedimiento que produce cada cambio (regla RN-03). Elaboración propia.

Paso	Etapa	Qué ocurre
1	Consulta	El cliente elige categoría o vehículo y revisa fechas libres con `fn_vehiculo_disponible`
2	Reserva	`sp_crear_reserva` verifica conflictos y crea la reserva en estado SOLICITADA
3	Confirmación	Un operador la valida con `sp_confirmar_reserva` y pasa a CONFIRMADA
4	Entrega	`sp_iniciar_alquiler` crea el alquiler (estado ACTIVA) y la inspección inicial se guarda en MongoDB
5	Devolución	Se registra el retorno y la inspección final; `sp_cerrar_alquiler` calcula cargos y genera el pago de prueba

Paso	Etapa	Qué ocurre
6	Cierre	La reserva queda CERRADA, el vehículo vuelve a DISPONIBLE y el historial queda consultable

4.3 Reglas de negocio

La más importante es RN-01. No depende solo de la interfaz: se implementa con una restricción de exclusión de PostgreSQL sobre un rango de fechas (`btree_gist`).

ID	Regla
RN-01	Un vehículo no puede tener dos reservas en estado SOLICITADA, CONFIRMADA o ACTIVA con períodos que se solapen.
RN-02	Un vehículo no puede reservarse en fechas que coincidan con un mantenimiento registrado.
RN-03	El estado de la reserva solo avanza en orden: SOLICITADA, CONFIRMADA, ACTIVA y CERRADA. CANCELADA solo es posible desde SOLICITADA o CONFIRMADA.
RN-04	El costo se calcula con la tarifa diaria vigente de la categoría. Cada día de retraso en la devolución genera un cargo del 50 % de esa tarifa (valor de demostración).
RN-05	Un alquiler solo se cierra después de registrar la devolución y su inspección.

5. Diseño de datos

5.1 Modelo relacional

El diseño incluye 13 entidades: Usuario, Rol, Usuario_Rol, Cliente, Categoria_Vehiculo, Vehiculo, Tarifa, Reserva, Alquiler, Pago, Cargo_Adicional, Mantenimiento y Auditoria_Reserva. Se normaliza hasta 3FN: la tarifa depende de la categoría y de su vigencia, y el costo y la condición de «reservado» se calculan a partir de las reservas en lugar de guardarse duplicados.

Tipo	Relación	Nota
1:1	Reserva – Alquiler	Un alquiler nace de exactamente una reserva (`id_reserva` único en Alquiler)
1:1	Usuario – Cliente	Un cliente con acceso web tiene un único usuario (relación opcional)
1:N	Categoria – Vehiculo; Categoria – Tarifa	Cada categoría agrupa varios vehículos y tiene tarifas con vigencia
1:N	Cliente – Reserva; Vehiculo – Reserva; Vehiculo – Mantenimiento; Alquiler – Pago; Alquiler – Cargo_Adicional	Historial de cada cliente, vehículo y alquiler
N:M	Usuario – Rol	Se resuelve con la tabla Usuario_Rol

5.2 Modelo documental en MongoDB

MongoDB tiene dos colecciones: `inspecciones_vehiculo` y `bitacora_operaciones`. Una inspección incluye el identificador del alquiler, el tipo (entrega o devolución), la fecha, el combustible, el kilometraje, una lista variable de daños (zona, severidad, descripción) y las evidencias referenciadas. La bitácora guarda eventos de la aplicación, como inicios de sesión e intentos de reserva rechazados.

¿Por qué NoSQL aquí? Las inspecciones tienen estructura variable, porque cada una puede tener cero o muchos daños y evidencias, y se consultan como documento completo, algo incómodo de modelar en tablas. La bitácora es de escritura continua y sin relaciones. En cambio, el estado oficial de las reservas, los pagos y la auditoría transaccional se quedan en PostgreSQL. El vínculo entre las dos bases es `id_alquiler`, que la aplicación verifica, porque no existen llaves foráneas entre motores.

Operaciones previstas: CRUD completo de inspecciones, una consulta compleja (daños de severidad alta en un rango de fechas, con `$elemMatch`) y tres agregaciones (kilometraje y combustible promedio por categoría, zonas de daño más frecuentes por mes con `$unwind` y `$group`, e incidencias por vehículo).

5.3 Lógica de negocio en PostgreSQL

Objeto	Nombre	Función
Procedimientos	`sp_crear_reserva`, `sp_confirmar_reserva`, `sp_iniciar_alquiler`, `sp_cerrar_alquiler`, `sp_cancelar_reserva`	Cada uno ejecuta una operación completa y controlada; la aplicación solo los invoca
Funciones	`fn_vehiculo_disponible`, `fn_calcular_costo`, `fn_calcular_cargo_retraso`	Consultan disponibilidad y calculan el valor según días, tarifa y retraso
Triggers	`trg_validar_transicion_reserva`, `trg_validar_mantenimiento`, `trg_auditar_reserva`, `trg_actualizar_estado_vehiculo`	Protegen RN-02 y RN-03, registran cambios en Auditoria_Reserva y sincronizan el estado del vehículo
Excepciones	`e_reserva_solapada`, `e_vehiculo_en_mantenimiento`, `e_transicion_invalida`	Errores propios con mensajes claros que la aplicación traduce a avisos
Transacciones	Dentro de cada `sp_`	La operación se confirma completa (`COMMIT`) o se revierte (`ROLLBACK`) ante un conflicto o error

5.4 Seguridad e integridad

- **Integridad:** llaves primarias y foráneas, `UNIQUE` (por ejemplo la placa), `CHECK` (fecha final posterior a la inicial, tarifas positivas) y la restricción de exclusión de RN-01.
- **Privilegios:** un rol de aplicación que ejecuta los `sp_` y consulta vistas, sin `DELETE` directo sobre reservas; un rol de solo lectura para reportes y un rol administrador.
- **Autenticación:** contraseñas con hash (BCrypt), autorización por perfil en Spring Security y un usuario de MongoDB limitado a su base.
- **Rendimiento:** índices `ix_` sobre vehículo y fechas de reserva y sobre las consultas frecuentes de reportes.

5.5 Reportes SQL previstos

Reporte	Técnica SQL
R1. Ocupación de cada vehículo por mes (días alquilados frente a días del mes)	`JOIN`, `GROUP BY`, `generate_series`
R2. Ingresos simulados por categoría y por mes	Agregaciones y vista
R3. Los cinco vehículos más alquilados de cada categoría	`RANK() OVER`
R4. Clientes con más reservas y su tasa de cancelación	Subconsultas y `CASE`
R5. Devoluciones tardías y cargos adicionales	`JOIN` y `HAVING`
R6. Vehículos disponibles en un rango de fechas	`NOT EXISTS` / `fn_vehiculo_disponible`

5.6 Virtualización y selección tecnológica

Se compara Ubuntu Server LTS, Debian y Rocky Linux (soporte, documentación, paquetes de PostgreSQL y MongoDB y consumo de recursos) y, para el motor relacional, PostgreSQL frente a MySQL y MariaDB (restricciones de exclusión, tipos de rango, PL/pgSQL y control transaccional). La hipótesis de trabajo es Ubuntu Server LTS con PostgreSQL, que se confirma con una matriz de criterios ponderados. En la VM, PostgreSQL, MongoDB y la aplicación corren como servicios de `systemd`, y se documentan los puertos, el arranque y las copias de respaldo con `pg_dump` y `mongodump`.

6. Requerimientos

La prioridad es Alta (indispensable), Media (importante) o Baja (deseable). La última columna dice qué objeto implementa cada requerimiento.

ID	Requerimiento	Prioridad	Trazabilidad
RF-01	Autenticación y perfiles (administrador, operador, cliente)	Alta	Usuario, Rol; Spring Security
RF-02	Consultar el catálogo por categoría y fechas libres	Alta	`fn_vehiculo_disponible`; R6
RF-03	Crear y cancelar reservas sin solapamientos	Alta	`sp_crear_reserva`; exclusión RN-01
RF-04	Confirmar reservas e iniciar alquileres	Alta	`sp_confirmar_reserva`; `sp_iniciar_alquiler`
RF-05	Registrar devolución, cargos y pago de prueba	Alta	`sp_cerrar_alquiler`; `fn_calcular_costo`
RF-06	Registrar inspecciones de entrega y devolución	Alta	MongoDB: `inspecciones_vehiculo`
RF-07	Consultar historial y reportes	Media	R1 a R6; agregaciones de MongoDB

ID	Requerimiento	Prioridad	Trazabilidad
RF-08	Registrar mantenimientos de vehículos	Media	Mantenimiento; `trg_validar_mantenimiento`
RNF-01	Sin dobles asignaciones, aun con accesos simultáneos	Alta	Restricción de exclusión; prueba de concurrencia
RNF-02	Seguridad: contraseñas con hash y privilegios mínimos	Alta	Roles de PostgreSQL; BCrypt
RNF-03	Despliegue reproducible y documentado en VM Linux	Alta	Guía de instalación y scripts
RNF-04	Consultas frecuentes en menos de 2 segundos con los datos de prueba	Media	Índices `ix_`

7. Gestión del proyecto

7.1 Metodología

Usamos una metodología híbrida. La Fase 1 (modelado) es secuencial, tipo cascada, porque el modelo E-R, el relacional y el diccionario tienen que cerrarse antes de implementar. La Fase 2 usa Scrum ligero, con sprints de una semana y una revisión conjunta al final de cada uno. El trabajo se gestiona en Jira con un tablero Scrum de cuatro sprints (SIRAV Sprint 1 a 4), cada uno con su meta. Un integrante actúa como Product Owner y el otro como Scrum Master, y los roles pueden rotar. Elegimos este enfoque porque el plazo es corto, el alcance está definido y hace falta detectar bloqueos a tiempo.

7.2 Cronograma

Semana	Trabajo central	Hito
1	Propuesta, supuestos, requerimientos, riesgos, Gantt y RACI; modelos E-R, Crow's Foot y relacional; diccionario de datos; análisis de SO y RDBMS; instalación de la VM	H1: modelado cerrado, gestión definida y entorno inicial listo
2	Scripts de creación e inserción; procedimientos, funciones, triggers, excepciones y transacciones; roles; documento de reportes SQL; modelo documental de MongoDB	H2: base relacional probada y entregables de la Fase 1 completos
3	MongoDB (CRUD, consultas complejas, agregaciones); catálogo, reservas y alquileres en la aplicación; conexión a ambas bases	H3: versión web integrada para pruebas
4	Pruebas de concurrencia y validaciones; reportes en la aplicación; informe ejecutivo; video; repositorio final	H4: prototipo verificable y entrega final

7.3 Matriz de riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
R1	Curva de aprendizaje de restricciones de exclusión y PL/pgSQL	Media	Alto	Prototipo en la semana 1; alternativa con trigger y `SELECT FOR UPDATE`

ID	Riesgo	Prob.	Impacto	Mitigación
R2	Recursos insuficientes del equipo para la VM	Media	Alto	Perfil mínimo de 4 GB; probar la VM en la semana 1
R3	Retraso al integrar Spring Boot con dos bases de datos	Media	Alto	Probar ambas conexiones en la semana 2, antes de la interfaz
R4	Reparto desigual del trabajo o integrante no disponible	Media	Alto	Revisión semanal, puntos de historia por persona en Jira y respaldo cruzado de roles
R5	Crecimiento del alcance con funciones nuevas	Alta	Medio	Alcance congelado; lo adicional pasa a ampliaciones futuras
R6	Pérdida de código o de la máquina virtual	Baja	Alto	Repositorio en GitHub, scripts de despliegue, instantánea de la VM y copias de las bases

7.4 Control de versiones

- Repositorio en GitHub con scripts SQL numerados, código de la aplicación, documentación y guía de la VM.
- Ramas `main` (estable), `develop` (integración) y `feature/` por tarea, con pull requests revisados por el otro integrante.
- Commits con prefijo (`feat` , `fix` , `docs` , `db`) y referencia a la tarea de Jira.

El repositorio está en github.com/DavidBejarano309/SIRAV.

8. Estado actual

Esta tabla resume el tablero de Jira al 3 de noviembre de 2026: 31 historias de usuario repartidas en siete épicas. Se actualiza cuando se publica una nueva versión del portafolio.

Épica	Terminadas	En curso	Por hacer
Gestión y documentación	3	0	2
Modelado y diseño de datos	0	2	1
Infraestructura Linux y VM	1	2	0
PostgreSQL: lógica de negocio	0	0	9
MongoDB	0	2	1
Aplicación web e integración	0	0	5
Calidad y pruebas	0	0	2
Total	**4**	**6**	**21**

Entregable	Estado
Propuesta del proyecto (v1.0)	Publicada: PDF · Word

Entregable	Estado
Supuestos, requerimientos, riesgos, Gantt y RACI	Terminado en Jira; supuestos, requerimientos, riesgos y cronograma están en la propuesta
Jira, tablero Scrum y repositorio con ramas	Terminado
Análisis de selección de SO y RDBMS	Terminado en Jira; documento por publicar
Modelos E-R, relacional y diccionario de datos	En elaboración
Instalación de la VM y guía de despliegue	En curso
Modelo documental y CRUD en MongoDB	En curso
Scripts de PostgreSQL, reportes SQL y seguridad	Por hacer; en el repositorio hay solo los archivos base
Aplicación web y pruebas	Por hacer
Informe ejecutivo y video demostrativo	Por hacer

Fuentes

El contenido de las secciones 1 a 7 sale de la propuesta del proyecto (versión 1.0, 17 de octubre de 2026) y del documento de supuestos del repositorio. La sección 8 sale del tablero de Jira del proyecto SIRAV. Los diagramas son de elaboración propia.

Referencias bibliográficas de la propuesta:

1. Bradshaw, S., Brazil, E., & Chodorow, K. (2019). *MongoDB: The definitive guide* (3.ª ed.). O'Reilly Media.
2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.
3. Sadalage, P. J., & Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.
4. Schwaber, K., & Sutherland, J. (2020). *La Guía de Scrum*. scrumguides.org
5. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7.ª ed.). McGraw-Hill.